0.问题的引入

程序通常是按指定的语法编写的好的，用来知名计算机该如何处理数据的规则，其在计算机科学中 的定义 是

$$\text{对某一个有限的数据集合所实施的，目的在于解决某一问题的一组有限指定的集合}$$

用一个公式可以很好的对程序的定义

$$\text{程序} = \text{数据结构} + \text{算法}$$

eg:

```
输入两个数据，计算这两个数的和
1.等待用户输入数据a,b
2.计算a+b
3.把数据写回
```

程序的执行方式

(1)顺序执行

```
一个程序完全指向完毕之后，才能执行下一个程序
eg:
    把一个程序分成三个步骤
        输入数据      ->   计算数据  -> 写回文件(打印出来)
    /*
        在这个过程中，计算数据的时候，cpu(中央处理器)才会被利用起来，而对于输入数据和写回文件的步骤，cpu是空闲状态
    */
    cpu利用率低
```

(2)并发执行(时间片轮转 ->分时系统)

```
把一个程序分成三个步骤
    输入数据      ->   计算数据  -> 写回文件(打印出来)
不同的步骤，由不同的硬件完成，这样就能同时运行多个程序
    硬件1       硬件2       硬件3
1. 输入数据  ->   计算数据  -> 写回文件(打印出来)
2. 输入数据  ->   计算数据  -> 写回文件(打印出来)
提高了cpu的利用率

并行:同一时刻执行多项任务，依赖的是多个cpu
并发:同一时间段执行多项任务
```

1.进程的概念

- 进程是一个程序的执行过程，当程序执行后，则进程会产生，执行过程结束了，则进程结束

> 进程是一个程序的执行过程，同一个程序每次运行的结果不一定一样

- 系统的每一个程序都会运行在某一个进程的上下文中

示例场景中有两个并发的进程：shell 进程和 hello 进程。最开始，只有 shell 进程在运行，即等待命令行上的输入。当我们让它运行 hello 程序时，shell 通过调用一个专门的函数，即系统调用，来执行我们的请求，系统调用会将控制权传递给操作系统。操作系统保存 shell 进程的上下文，创建一个新的 hello 进程及其上下文，然后将控制权传给新的 hello 进程。hello 进程终止后，操作系统恢复 shell 进程的上下文，并将控制权传回给它，shell 进程会继续等待下一个命令行输入。

如图 1-12 所示，从一个进程到另一个进程的转换是由操作系统内核(kernel)管理的。内核是操作系统代码常驻主存的部分。当应用程序需要操作系统的某些操作时，比如读写文件，它就执行一条特殊的系统调用(system call)指令，将控制权传递给内核。然后内核执行被请求的操作并返回应用程序。注意，内核不是一个独立的进程。相反，它是系统管理全部进程所用代码和数据结构的集合。

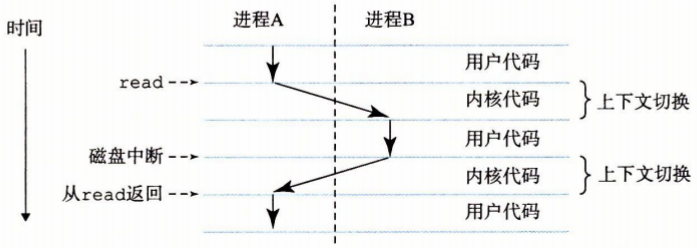
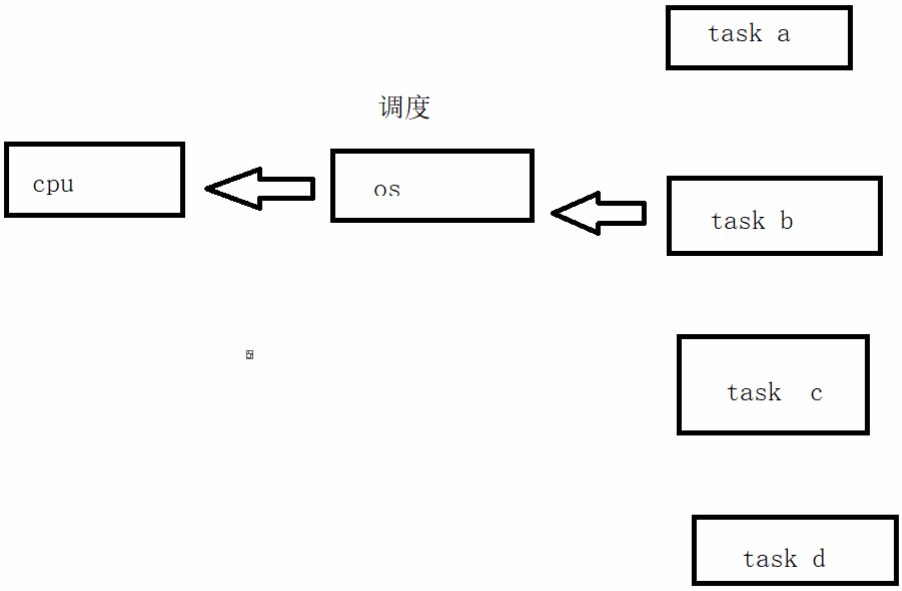


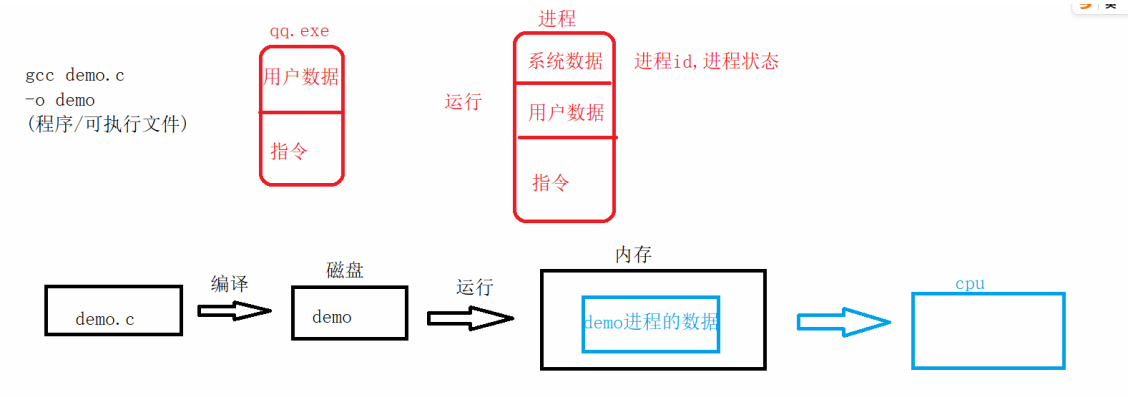
图 1-12　进程的上下文切换

- 进程的特点

  1.进程是一个独立的可调度的活动，由操作系统进行统一的管理，相应的任务就会被cpu调度



2.进程一旦产生，则需要分配相关的资源，同时进程是分配资源的最小单位

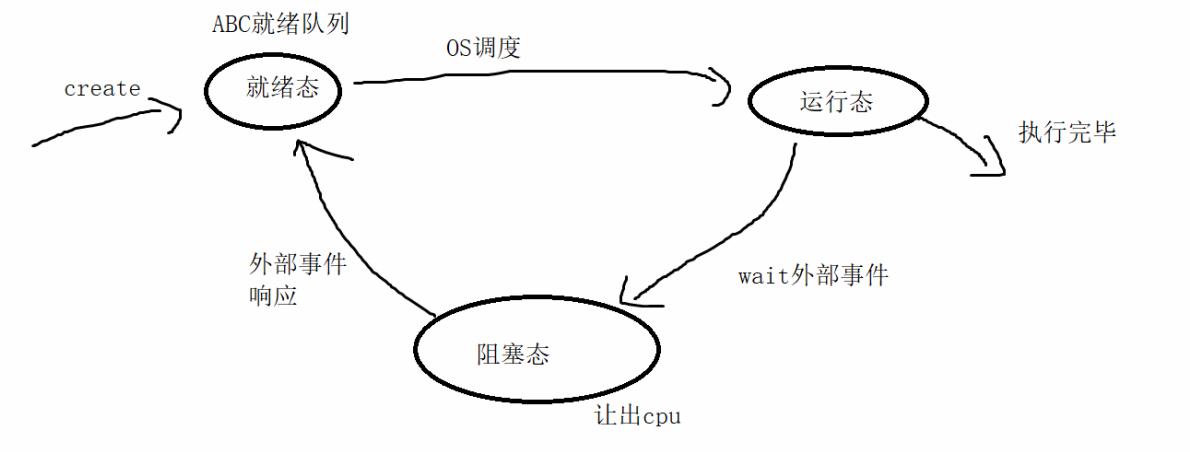


2.进程和程序的区别

- 程序是静态的概念，进程是动态的概念
- 进程是一个程序的一次执行活动
- 进程是一个独立的活动单位，是竞争资源的基本单位

3.进程的状态

```
1.可执行程序是存放在硬盘，运行需要把程序放到内存，进程正在被创建
    操作系统会为进程分配资源
2.当进程完成创建后，会进入到就绪态
    已经具备了运行的条件，但是由于没有空闲的cpu(为别的进程服务)，暂时不能运行
3.当cpu空闲，操作系统就会选择一个没有就绪的程序，让它上cpu
    如果一个进程此时在cpu运行，就是处于运行态
4.进程运行过程中，可能会请求等待某个外部事件的发生(如等待某种系统资源的分配，等待其他的进程的响应)
    阻塞态(睡眠态)
5.一个进程结束了，进入到终止态(让进程下cpu,并且回收内存资源)
```



拓展:一般会把操作系统分成两类

```
1.分时系统
    以时间片轮转为主，每个进行执行一段时间(时间片)
    eg:
        大部分的桌面系统
        linux,windows,IOS,MACOS,安卓....
2.实时系统
    调度的策略主要以实时策略为主
    实时策略:
        每次调度都取优先级最高的那个进程执行，直到该进程执行完毕/主动放弃cpu/有更高的优先级进程(抢占/插队)
    eg:
        ucos/freeRTOS
```

4.linux进程地址空间

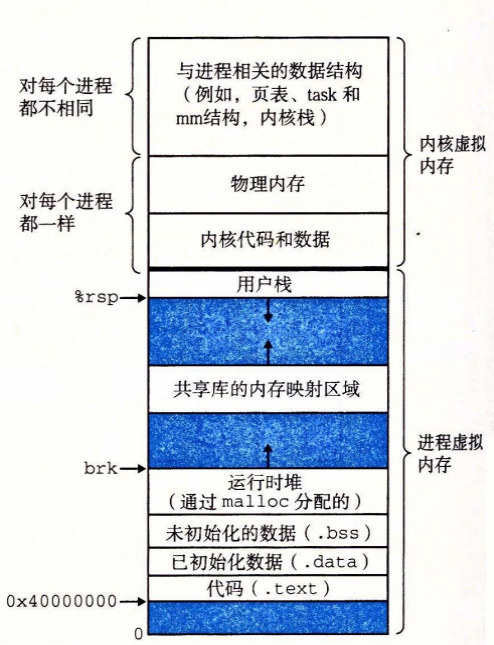
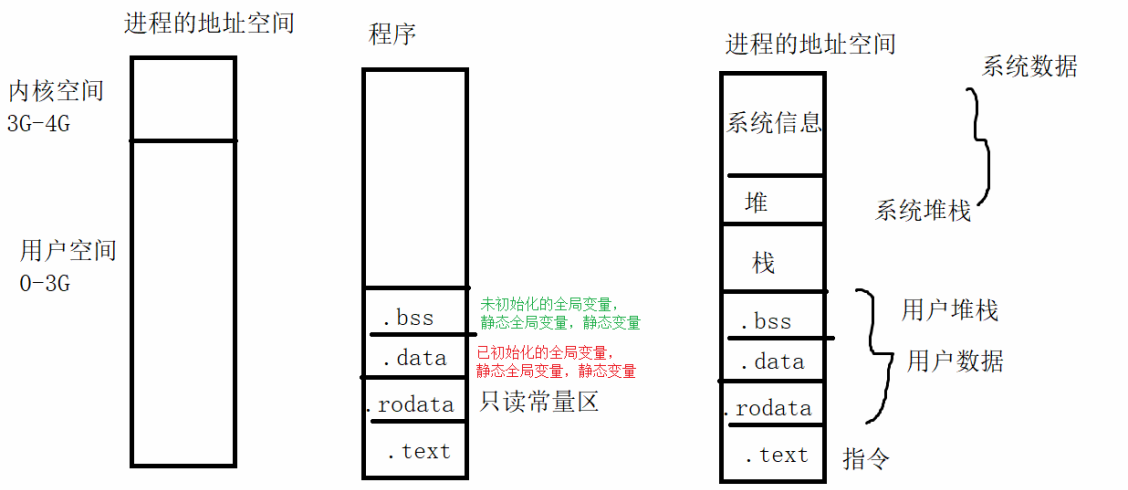


图 9-26　一个 Linux 进程的虚拟内存

- 一旦进程创建好后，系统则要为这个进程分配相应的资源，一般系统会每个进程分配4G的地址(32位系统)
- 4G地址空间结构

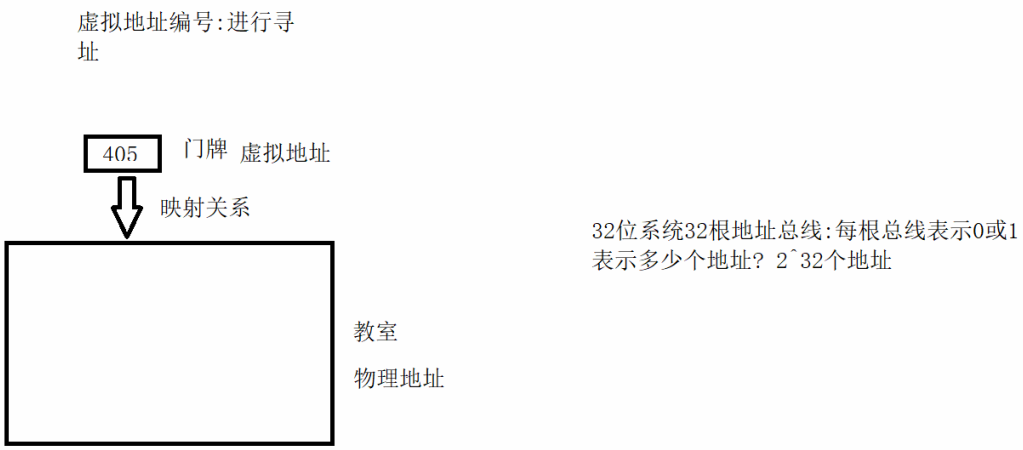


为什么要分成用户空间和内核空间?
　　为了保护内核的安全，任何人都能修改内核的数据，行为很危险，会造成系统的奔溃
　　当用户进程需要内核获取资源的时候(例如获取磁盘中的数据)，会切换成内核态运行，这就会使当前的进程获取内核空间的资源

- 会为进程分配4G的虚拟地址空间

虚拟地址空间中每一个地址都是虚拟地址

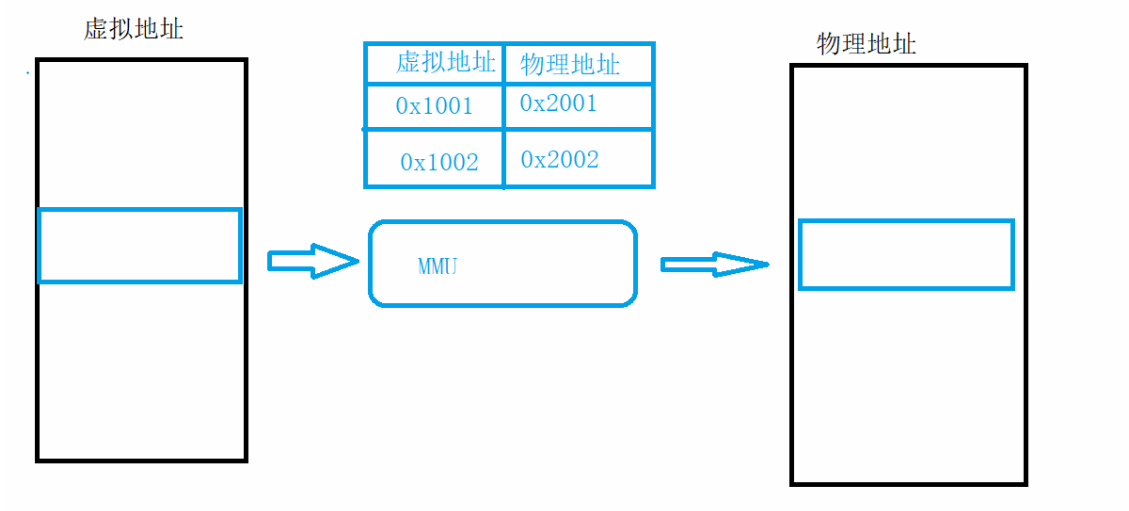
虚拟地址:并不是真的内存空间，是用来寻址的编号
物理地址:内存设备中真实存在的存储空间中编号



- 虚拟地址通过映射的方式建立物理地址的关系，从而达到访问虚拟地址就可以访问到对应的物理地址

在cpu中有个硬件MMU(内存管理单元)，负责虚拟地址与物理地址的映射管理以及虚拟地址访问

使用虚拟地址原因:
　　直接访问物理地址，会导致地址没有隔离，很容易导致数据的被修改
　　每个进程的虚拟地址空间相互独立的，操作系统会映射到不同的物理地址区间中，再访问的时候就不会相互干扰
　　物理地址使用有限



5.进程的创建

进程号类型为pid_t

调用fork函数，则会产生一个新的进程

调用fork的那个进程叫做父进程，产生的新的进程称为子进程

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);
返回值:
    成功
        子进程返回值0
        父进程返回子进程的id  >0
    失败
        返回-1
```

参考示例

```
#include <sys/types.h>
#include <unistd.h>

int main()
{
    fork();
    fork();
    printf("hello\n");
}

结果打印4次hello
fork两次一共有4个进程，每个进程都打印一次hello
```

5.1区分父子进程

注意调用一次fork会有两个返回
两次返回值的区别，子进程返回0 父进程返回子进程id

getpid：获取自己的进程号

getppid:获取调用这个函数的进程的父进程号

eg:子进程循环打印aaaa,每隔一秒打印一次 父进程循环打印bbbb,每隔一秒打印一次

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        //子进程
        printf("子: %d\n",getpid());
        printf("父: %d\n",getppid());
        // while(1)
        // {
        //     printf("aaaa\n");
        //     sleep(1);
        // }
    }
    else if(pid > 0)
    {
        printf("pid = %d\n",pid);//打一下pid的值 看一下是否跟子进程id一致
        //父进程
        // while(1)
        // {
        //     printf("bbbb\n");
        //     sleep(1);
        // }
    }
}
```

5.2父子进程的关系

```
一个进程里面包含哪些内容?
-系统数据
-数据(用户数据)
-指令(代码)

fork这个函数在创建子进程的时候，拷贝父进程的数据和指令
    整个进程地址空间中：父进程的变量，数据对象，标准IO缓冲区，文件描述符，进程的堆栈....
    copy了父进程的所有(目前可以这么认为)

子进程所独有的，只有它的进程号，进程的状态,...(系统数据)

使用fork函数代价比较大
```

5.3父子进程地址空间

父子进程地址空间是相互独立的

创建一个子进程，并定义一个全局变量global = 0,在子进程中修改值为100，在父进程中打印global的值，思考为什么是这样的结果

```
#include <sys/types.h>
```

```c
#include <unistd.h>
#include <stdio.h>
int global = 0;
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        //子进程
        global = 100;
        while(1)
        {
            printf("子: %d %p\n",global,&global);
            sleep(1);
        }
    }
    else if(pid > 0)
    {
        //父进程
        while(1)
        {
            printf("父: %d %p\n",global,&global);
            sleep(1);
        }

    }
}
```

思考

1.fork一旦成功，父子进程先执行哪一个?

```
操作系统调度
```

2.fork之后，子进程拷贝父进程的数据和指令，到底拷贝了什么数据?

```
-父进程全部的用户数据(局部变量 全局变量 静态变量...)
-标准IO缓冲区
-父进程打开的文件描述符和状态
-信号处理的方式(进程间通信)
-...
```

3.分析结果

```
int main()
{
    printf("abc");
    fork();
}
//abcabc

int main()
{
    printf("abc\n");
    fork();
}
//abc换行
```

6.进程的退出

进程退出两种情况

- 自杀(自己退出)
  - main函数的返回退出结束　会做清理工作，会把缓冲区的内容同步到终端

    用法：return n;
  - 进程执行exit 结束进程　会做清理工作，会把缓冲区的内容同步到终端

    头文件：#include <stdlib.h>　标准库函数

    用法：exit(n);
  - 进程执行_exit/_Exit 结束进程　不会做清理工作，会把缓冲区的内容直接丢弃

    头文件：#include <unistd.h>　linux系统调用函数

    用法：_exit(n);
- 它杀(操作系统)

  一个进行 可以 杀死另外一个进程(信号)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main()
{
    printf("abc");
    #if 0
    _exit(0);//等价于 _Exit(0)  不会做清理工作，会把缓冲区的内容直接丢弃
    #else
    exit(0);//等价于 return 0;  会做清理工作，会把缓冲区的内容输出到终端
    #endif
}
```

7.等待子进程退出

wait/waitpid：用来回收子进程的资源

这两个函数用来等待某个子进程退出状态发生改变的，等待的状态发生变化有三种情况

- 子进程退出(正常退出)，main函数返回return/exit/_exit
- 子进程被信号终止
- 子进程被信号唤醒(blocking阻塞->ready)

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *wstatus);
@wstatus:指向的空间，保存子进程的退出信息(怎么死的，退出码)
返回值：
    成功：返回退出的那个进程的id
    失败：返回-1

eg:不想知道子进程怎么死的，只需要回收子进程的资源
    wait(NULL);
eg:想知道怎么结束子进程的
    int wstatus;
    wait(&wstatus);

    wstatus用来保存退出的那个子进程的退出信息的，退出信息保存到一个整数里面
    可以用以下宏解释子进程的退出信息
        WIFEXITED(wstatus)
            如果子进程正常终止(exit/_exit或main返回)，则返回真
        WEXITSTATUS(wstatus)
            返回子进程退出状态(退出码)，只有当WIFEXITED(wstatus)为真的时候，才能用这个宏
        WIFSIGNALED(wstatus)
            如果子进程因信号而终止，则返回真
        WTERMSIG(wstatus)
            返回导致子进程终止信号的编号，只有当WIFSIGNALED(wstatus)为真的时候，才能用这个
宏
```

练习：创建子进程，子进程先打印自己的pid，接着循环1s打印一次数据aaa，父进程使用wait函数等待子进程结束，判断子进程是否是因信号而终止，如果是，则打印终止的那个信号的编号

把这个程序写完后,打开终端，运行，记住打印的pid,接着打开一个新的终端，输入 kill -9 子进程pid

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        //子进程
        printf("子：%d\n",getpid());
        while(1)
        {
            printf("aaa\n");
            sleep(1);
        }
    }
    else if(pid > 0)
```

```
    {
        //父进程
        //等待子进程结束
        int wstatus;
        wait(&wstatus);

        if(WIFEXITED(wstatus))//判断是否正常结束子进程
        {
            //打印退出码
            printf("exit code:%d\n",WEXITSTATUS(wstatus));
        }
        if(WIFSIGNALED(wstatus))//判断子进程是否因信号而终止
        {
            printf("signal number:%d\n",WTERMSIG(wstatus));
        }
    }
}
```

waitpad

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *wstatus, int options);
@pid:指定要等待的进程/进程组
    pid < -1
        表示等待进程组 ID等于该 pid 绝对值的任意子进程。
        -1234:表示等待1234那个进程组的所有子进程
    pid = -1
        表示等待任意子进程

    pid = 0
        表示等待进程组 ID 与调用进程相同的任意子进程。

    pid > 0
        表示等待进程 ID 等于该 pid 值 的子进程。
@wstatus:指向的空间，用来保存子进程退出信息
@options：等待选项
    0           默认阻塞等待
    WNOHANG     非阻塞等待，假如没有子进程退出，则立即返回
返回值：
    成功：返回退出的那个进程的id
    失败：返回-1

eg:
    wait(&wstatus) <==> waitpid(-1,&wstatus,0)
```

进程组

```
就是一组进程，每个进程会属于某个进程组
并且每个进程组，都会有一个组进程
创建这个进程组的进程就是组长，经常组有一个组id(组长的pid)

A创建B和C，B创建D
进程创建的关系
    A(pid=1000,pgid=1000)
```

```
        B(pid=1001,pgid=1000) 默认继承关系
            D(pid=1003,pgid=1000) 默认继承关系
        C(pid=1002,pgid=1000) 默认继承关系
进程创建的关系
    A(pid=1000,pgid=1000)
        B(pid=1001,pgid=1001) <--新进程组 setpgid
            D(pid=1003,pgid=1001) <--继承B进程组
        C(pid=1002,pgid=1000) <--继承A进程组
```

练习：子进程立即结束，父进程一直死循环，不调用wait,建议把子进程和父进程的id打印

写完代码运行后，打开新的终端，使用ps -aux | grep Z 查看进程，试一下，能不能把子进程杀死

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        //子进程
        printf("子:%d\n",getpid());
        exit(0);
    }
    else if(pid > 0)
    {
        //父进程
        printf("父:%d\n",getpid());
        while(1);

    }
}
```

僵尸进程：

```
子进程终止，父进程没有回收子进程的资源，子进程就会变成僵尸进程
```

孤儿进程

```
父进程运行结束，但是子进程还在运行，这个子进程就会变成孤儿进程
```

8.exec函数簇

eg:另外一个程序

```c
#include <stdio.h>
#include <stdlib.h>

//gcc 08-test.c -o 08-test
//./08-test 3 4
int main(int argc,char **argv)
{
    printf("%s + %s = %d\n",argv[1],argv[2],atoi(argv[1])+atoi(argv[2]));
}
```

进程替换

```
exec函数簇 是让一个程序去执行另外一个程序的文件

exec让一个进程去执行另外一个程序文件
    需要指定这个程序文件的名字
        一个文件系统中程序文件的名字(带路径)
    还得指定程序运行所需要的参数
        对于linux程序的参数，都是当成字符串来处理的

    指定参数的方式有两种
        l:list
            把程序运行的参数，一个一个列举出来
            "08-test","3","4",NULL
        v:vector 向量(数组)
            把程序运行的参数，弄成一个char*数组
            char *arg[] = {"08-test","3","4",NULL};
```

```c
#include <unistd.h>

extern char **environ;

int execl(const char *path, const char *arg, ...
/* (char  *) NULL */);
@path:你要执行的程序文件的名字(带路径)
@arg,...
    程序运行的参数  list->以列表的方式指定
返回值:
    成功 永远都不返回!!!因为当前的进程的整个数据和指令都被别人替换了，没得返回
    失败 -1

int execlp(const char *file, const char *arg, ...
/* (char  *) NULL */);
@path:你要执行的程序文件的名字(不带路径)(会在PATH路径下找)
@arg,...
    程序运行的参数  list->以列表的方式指定
返回值:
    成功 永远都不返回!!!因为当前的进程的整个数据和指令都被别人替换了，没得返回
    失败 -1
int execv(const char *path, char *const argv[]);
@path:你要执行的程序文件的名字(带路径)
@argv
    程序运行的参数  v->以数组的方式指定
返回值:
    成功 永远都不返回!!!因为当前的进程的整个数据和指令都被别人替换了，没得返回
    失败 -1
```

```
int execvp(const char *file, char *const argv[]);
@path:你要执行的程序文件的名字(不带路径)(会在PATH路径下找)
@arg,...
    程序运行的参数   v->以数组的方式指定
返回值:
    成功 永远都不返回!!!因为当前的进程的整个数据和指令都被别人替换了，没得返回
    失败 -1
```

eg:

在子进程中查看当前目录下的文件详细信息

在终端执行命令

```
ls -l
```

=>ls程序文件实在PATH指定的目录下"/bin/ls"

```
execl("/bin/ls","ls","-l",NULL);
execl("/mnt/hgfs/CS2612/三阶段/01-并发/01-进程/02-code
/08-test","./08-test","3","4",NULL);
```

或

```
execlp("ls","ls","-l",NULL);//ls的/bin路径需要在PATH路径中
execlp("08-test","./08-test","3","4",NULL);

添加环境变量
export PATH=$PATH:你的路径
```

使用execv和execvp实现程序

9.system

作业:

1.程序执行完成后，输出的结果是多少,分析原因

```
int main()
{
    int i;
    for(i = 0;i<2;i++)
    {
        fork();
        printf("_\n");
    }
}
```

2.程序执行完成后，输出的结果是多少,分析原因

```
int main()
{
    int i;
    for(i = 0;i<2;i++)
    {
        fork();
        printf("_");
    }
}
```

3.写一个madplay的程序，播放一个指定目录下所有的mp3文件

```
sudo apt install madplay 要联网
```